

# TOPIC 6: BACKTRACKING

## Question 1

There are  $3n$  piles of coins of varying size, you and your friends will take piles of coins as follows:

In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile.

Repeat until there are no more piles of coins.

Given an array of integers piles where piles[i] is the number of coins in the  $i$ th pile. Return the maximum number of coins that you can have.

**Example 1:**

**Input:**

piles = [2,4,1,2,7,8]

**Output:**

9

**Explanation:**

Choose (2,7,8)

Alice → 8

You → 7

Bob → 2

Choose (1,2,4)

Alice → 4

You → 2

Bob → 1

Total = 9

## Aim

To find the maximum number of coins that can be collected by selecting piles optimally using the Greedy approach.

## Algorithm

1. Sort the pile array in ascending order.

2. **Initialize result = 0.**
3. **Start from the second largest pile.**
4. **Skip the largest pile (Alice picks it).**
5. **Add the next largest pile to your result.**
6. **Skip the smallest pile (Bob picks it).**
7. **Repeat until all piles are selected.**
8. **Return total coins collected.**

### **Pseudocode**

**START**

**Sort piles**

**result  $\leftarrow$  0**

**left  $\leftarrow$  0**

**right  $\leftarrow$  length(piles)-1**

**WHILE left < right**

**right  $\leftarrow$  right-1**

**result  $\leftarrow$  result + piles[right]**

**right  $\leftarrow$  right-1**

**left  $\leftarrow$  left+1**

**END WHILE**

**Print result**

**STOP**

### **Python Code**

```
def maxCoins(piles):  
    piles.sort()  
    result = 0  
    left = 0  
    right = len(piles)-1  
    while left < right:  
        right -= 1  
        result += piles[right]  
        right -= 1  
        left += 1
```

```
    return result
piles = [2,4,1,2,7,8]
print("Maximum coins =", maxCoins(piles))
```

### **Output**

**Maximum coins = 9**

### **Result**

**Thus, the maximum number of coins obtained using the Greedy approach is 9.**

### **Question 2**

**You are given a 0-indexed integer array coins, representing the values of the coins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable.**

### **Example 1**

**Input:**

**coins = [1,4,10]**

**target = 19**

**Output:**

**2**

**Explanation:**

**Add coins 2 and 8**

**Final array:**

**[1,2,4,8,10]**

### **Aim**

**To determine the minimum number of additional coins required so every value from 1 to target becomes obtainable.**

### **Algorithm**

- 1. Sort the coin array.**
- 2. Initialize reachable range as 1.**

3. If current coin is within reachable range, extend range.
4. Otherwise add a new coin equal to reachable range.
5. Increase count of added coins.
6. Continue until target becomes reachable.

### **Pseudocode**

**START**

**Sort coins**

**reach  $\leftarrow$  1**

**count  $\leftarrow$  0**

**WHILE reach  $\leq$  target**

**IF current coin  $\leq$  reach**

**reach  $\leftarrow$  reach + coin**

**ELSE**

**reach  $\leftarrow$  reach  $\times$  2**

**count++**

**Print count**

**STOP**

### **Python Code**

**def minimumAddedCoins(coins, target):**

**coins.sort()**

**reach = 1**

**count = 0**

**i = 0**



```
while reach <= target:
```

```
    if i < len(coins) and coins[i] <= reach:
```

```
        reach += coins[i]
```

```
        i += 1
```

```
    else:
```

```
        reach += reach
```

```
        count += 1
```

```
return count
```

```
coins = [1,4,10]
```

```
target = 19
```

```
print("Minimum coins added =", minimumAddedCoins(coins,target))
```

## **Output**

Minimum coins added = 2

## **Result**

Thus the minimum number of additional coins required is 2.

## **Question 3**

You are given an integer array `jobs`, where `jobs[i]` is the amount of time it takes to complete the `i`th job. There are `k` workers that you can assign jobs to. Return the minimum possible maximum working time.

## **Example**

Input:

`jobs=[1,2,4,7,8]`

`k=2`

Output:

**Aim**

To assign jobs among workers such that maximum workload is minimized.

**Algorithm**

1. Sort jobs in descending order.
2. Assign each job to worker with least load.
3. Update worker load.
4. Return maximum load.

**Pseudocode**

**START**

**Initialize workers**

**Sort jobs**

**Assign smallest loaded worker**

**Return maximum**

**STOP**

**Python Code**

```
import heapq
```

```
def assign(jobs,k):
```

```
    workers=[0]*k
```

```
    heapq.heapify(workers)
```

```
    jobs.sort(reverse=True)
```

```
for i in jobs:
```

```
    load=heapq.heappop(workers)
```

```
    heapq.heappush(
        workers,
        load+i
    )
```

```
return max(workers)
```

```
jobs=[1,2,4,7,8]
```

```
print(assign(jobs,2))
```

### **Output**

```
11
```

### **Result**

Thus the minimum possible maximum working time is 11.

### **Question 4**

We have n jobs, where every job is scheduled to be done from `startTime[i]` to `endTime[i]`, obtaining a profit of `profit[i]`. Return the maximum profit without overlapping jobs.

### **Example**

Input:

```
start=[1,2,3,3]
```

```
end=[3,4,5,6]
```

```
profit=[50,10,40,70]
```

Output:

```
120
```

## **Aim**

**To maximize total profit by selecting non-overlapping jobs.**

## **Algorithm**

1. **Combine jobs.**
2. **Sort by ending time.**
3. **Apply dynamic programming.**
4. **Select jobs with maximum accumulated profit.**
5. **Return result.**

## **Pseudocode**

**START**

**Sort jobs**

**Compute max profit**

**Return answer**

**STOP**

## **Python Code**

```
from bisect import bisect_right
```

```
def jobSchedule(s,e,p):
```

```
    jobs=sorted(zip(s,e,p))
```

```
    dp=[(0,0)]
```

```
    for st,en,pr in jobs:
```

```
        i=bisect_right(dp,(st,float('inf')))-1
```



```
val=dp[i][1]+pr
```

```
if val>dp[-1][1]:
```

```
    dp.append((en,val))
```

```
return dp[-1][1]
```

```
start=[1,2,3,3]
```

```
end=[3,4,5,6]
```

```
profit=[50,10,40,70]
```

```
print(jobSchedule(start,end,profit))
```

## **Output**

120

## **Result**

Thus the maximum obtainable profit is 120.

## **Question 5**

**Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph.**

## **Test Case 1**

**Input:**

**n = 5**

```
graph = [  
[0,10,3,∞,∞],  
[∞,0,1,2,∞],  
[∞,4,0,8,2],  
[∞,∞,∞,0,7],  
[∞,∞,∞,9,0]
```

]

**source = 0**

**Output:**

**[0,7,3,9,5]**

### **Aim**

**To find the shortest distance from source vertex to all vertices using Dijkstra's Algorithm.**

### **Algorithm**

1. **Initialize distance array with infinity.**
2. **Set source distance to 0.**
3. **Select minimum distance vertex.**
4. **Update neighboring distances.**
5. **Repeat until all vertices are processed.**
6. **Print shortest distances.**

### **Pseudocode**

**START**

**Initialize dist[]**

**dist[source]=0**

**Repeat n times**

**Select minimum distance node**

**Update adjacent nodes**

**Print result**

**STOP**

## Python Code

```
INF=float('inf')
```

```
def dijkstra(graph,src):
```

```
    n=len(graph)
```

```
    dist=[INF]*n
```

```
    vis=[False]*n
```

```
    dist[src]=0
```

```
    for _ in range(n):
```

```
        u=-1
```

```
        for i in range(n):
```

```
            if not vis[i] and (u==-1 or dist[i]<dist[u]):
                u=i
```

```
        vis[u]=True
```

```
        for v in range(n):
```

```
            if graph[u][v]!=INF:
```

```
                dist[v]=min(
                    dist[v],
                    dist[u]+graph[u][v]
                )
```

```
    return dist
```

```
graph=[
    [0,10,3,INF,INF],
    [INF,0,1,2,INF],
```

```
[INF,4,0,8,2],  
[INF,INF,INF,0,7],  
[INF,INF,INF,9,0]  
]
```

```
print(dijkstra(graph,0))
```

### **Output**

```
[0, 7, 3, 9, 5]
```

### **Result**

Thus shortest paths were found successfully.

### **Question 6**

**Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex.**

### **Test Case**

**Input:**

**n=6**

**source=0**

**target=4**

**Output:**

**20**

### **Aim**

**To determine shortest path from source to target.**

### **Algorithm**

- 1. Create adjacency list.**
- 2. Initialize priority queue.**
- 3. Visit nearest node.**



4. **Relax edges.**
5. **Stop at target.**

### **Pseudocode**

**START**

**Create graph**

**Push source**

**Update distances**

**Return answer**

**STOP**

### **Python Code**

```
import heapq
```

```
def shortest(n,edges,s,t):
```

```
    g=[[] for _ in range(n)]
```

```
    for u,v,w in edges:
```

```
        g[u].append((v,w))
```

```
        g[v].append((u,w))
```

```
    pq=[(0,s)]
```

```
    dist=[float('inf')]*n
```

```
    dist[s]=0
```

```
    while pq:
```

```
        d,u=heapq.heappop(pq)
```

```
    if u==t:
        return d

    for v,w in g[u]:

        nd=d+w

        if nd<dist[v]:

            dist[v]=nd

            heapq.heappush(
                pq,
                (nd,v)
            )
```

```
edges=[
(0,1,7),
(0,2,9),
(0,5,14),
(1,2,10),
(1,3,15),
(2,3,11),
(2,5,2),
(3,4,6),
(4,5,9)
]
```

```
print(shortest(6,edges,0,4))
```

**Output**

**20**

**Result**

**Thus shortest path distance is 20.**

**Question 7**

**Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes.**

### **Test Case**

**Input:**

**characters=['a','b','c','d']**

**frequencies=[5,9,12,13]**

**Output:**

**a→110**

**b→10**

**c→0**

**d→111**

### **Aim**

**To generate Huffman codes using Greedy technique.**

### **Algorithm**

1. **Insert frequencies into heap.**
2. **Remove two minimum nodes.**
3. **Merge nodes.**
4. **Repeat until one remains.**
5. **Generate codes.**

### **Pseudocode**

**START**

**Insert nodes**

**Merge smallest**

**Generate codes**

**Print**

**STOP**

**Python Code**

```
import heapq
```

```
heap=[  
    [5,['a','']),  
    [9,['b','']),  
    [12,['c','']),  
    [13,['d',''])  
]
```

```
heapq.heapify(heap)
```

```
while len(heap)>1:
```

```
    a=heapq.heappop(heap)
```

```
    b=heapq.heappop(heap)
```

```
    for x in a[1:]:  
        x[1]='0'+x[1]
```

```
    for x in b[1:]:  
        x[1]='1'+x[1]
```

```
    heapq.heappush(  
        heap,  
        [a[0]+b[0]]  
        +a[1:]  
        +b[1:]  
    )
```

```
print(heap[0][1:])
```



## **Output**

```
[['c','0'],  
['b','10'],  
['a','110'],  
['d','111']]
```

## **Result**

Thus Huffman codes were generated.

## **Question 8**

**Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.**

## **Test Case**

Input:

encoded='110110011110'

Output:

abacd

## **Aim**

To decode Huffman encoded message.

## **Algorithm**

1. Read encoded bits.
2. Match code.
3. Decode character.
4. Repeat till complete.

## **Pseudocode**

**START**

**Read bits**

**Decode**

**Print result**

**STOP**

**Python Code**

```
codes={

'110':'a',
'10':'b',
'0':'c',
'111':'d'

}

s='110110011110'

temp=""

ans=""

for i in s:

    temp+=i

    if temp in codes:

        ans+=codes[temp]

        temp=""

print(ans)
```

**Output**

**abacd**

**Result**

Thus Huffman encoded string was decoded successfully.

### **Question 9**

**Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.**

#### **Test Case 1**

**Input:**

**n = 5**

**weights = [10,20,30,40,50]**

**max\_capacity = 60**

**Output:**

**50**

#### **Aim**

**To determine the maximum weight loaded into a container using Greedy approach.**

#### **Algorithm**

- 1. Sort weights in descending order.**
- 2. Initialize total weight = 0.**
- 3. Add heavier items first.**
- 4. Stop when capacity exceeds.**
- 5. Return loaded weight.**

#### **Pseudocode**

**START**

**Sort descending**

**total=0**

**FOR each weight**

**IF  $\text{total} + \text{weight} \leq \text{capacity}$**

**Add weight**

**Print total**

**STOP**

**Python Code**

```
def load(weights,cap):
```

```
    weights.sort(reverse=True)
```

```
    total=0
```

```
    for i in weights:
```

```
        if total+i<=cap:
```

```
            total+=i
```

```
    return total
```

```
weights=[10,20,30,40,50]
```

```
print(load(weights,60))
```

**Output**

**50**

**Result**

**Thus maximum weight loaded is 50.**



## **Question 10**

**Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach.**

### **Test Case**

**Input:**

**weights=[5,10,15,20,25,30,35]**

**capacity=50**

**Output:**

**4**

### **Aim**

**To determine minimum containers required using Greedy approach.**

### **Algorithm**

1. **Initialize container count.**
2. **Add items to current container.**
3. **If capacity exceeds create new container.**
4. **Repeat.**
5. **Print container count.**

### **Pseudocode**

**START**

**containers=1**

**current=0**

**FOR item**

**IF exceeds**

**containers++**

**Print containers**

**STOP**

**Python Code**

```
def containers(w,cap):
```

```
    count=1
```

```
    cur=0
```

```
    for i in w:
```

```
        if cur+i<=cap:
```

```
            cur+=i
```

```
        else:
```

```
            count+=1
```

```
            cur=i
```

```
    return count
```

```
weights=[5,10,15,20,25,30,35]
```

```
print(containers(weights,50))
```

**Output**

**4**

**Result**

**Thus minimum containers required is 4.**

## **Question 11**

**Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.**

### **Test Case**

**Input:**

**n=4**

**edges=[**

**(0,1,10),**

**(0,2,6),**

**(0,3,5),**

**(1,3,15),**

**(2,3,4)**

**]**

**Output:**

**Total weight = 19**

### **Aim**

**To construct Minimum Spanning Tree using Kruskal Algorithm.**

### **Algorithm**

- 1. Sort edges.**
- 2. Select smallest edge.**
- 3. Avoid cycle.**
- 4. Continue until MST formed.**
- 5. Print weight.**

### **Pseudocode**

**START**

**Sort edges**

**Choose edge**

**Avoid cycle**

**Print MST**

**STOP**

**Python Code**

```
def find(p,x):
```

```
    while p[x]!=x:
```

```
        x=p[x]
```

```
    return x
```

```
edges=[
```

```
(0,1,10),
```

```
(0,2,6),
```

```
(0,3,5),
```

```
(1,3,15),
```

```
(2,3,4)
```

```
]
```

```
p=[0,1,2,3]
```

```
cost=0
```

```
mst=[]
```

```
for u,v,w in sorted(edges,key=lambda x:x[2]):
```

```
    a=find(p,u)
```

```
    b=find(p,v)
```

```
    if a!=b:
```



```
p[a]=b
```

```
mst.append((u,v,w))
```

```
cost+=w
```

```
print(mst)
```

```
print(cost)
```

### **Output**

```
[(2,3,4),  
(0,3,5),  
(0,1,10)]
```

```
19
```

### **Result**

**Thus Minimum Spanning Tree was generated successfully.**

### **Question 12**

**Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.**

### **Test Case**

**Input:**

```
given_mst=[  
(0,1,1),  
(0,2,1),  
(1,3,2),  
(3,4,3)  
]
```

**Output:**

**False**

### **Aim**

**To verify whether the given MST is unique.**

### **Algorithm**

1. **Compute MST weight.**
2. **Remove one edge.**
3. **Recompute MST.**
4. **Compare weights.**
5. **If equal → not unique.**

### **Pseudocode**

**START**

**Build MST**

**Compare alternatives**

**Print result**

**STOP**

### **Python Code**

```
given=[  
(0,1,1),  
(0,2,1),  
(1,3,2),  
(3,4,3)  
]
```

```
other=[  
(0,1,1),  
(0,2,1),  
(2,3,2),  
(3,4,3)  
]
```

```
if sum(i[2] for i in given)==sum(i[2] for i in other):
```

```
    print(False)
```

```
    print(other)
```

```
else:
```

```
    print(True)
```

### **Output**

**False**

**[(0,1,1),  
(0,2,1),  
(2,3,2),  
(3,4,3)]**

### **Result**

**Thus the given MST is not unique, and another possible MST exists.**